

REACT DEVELOPMENT SUMMER INTERNSHIP REPORT

Submitted by

TANVI
(02521402024)

in partial fulfillment for the award of the degree

of

Bachelor of Computer Application (2024-28)



Jagannath International Management School

Vasant Kunj, New Delhi – 110070

(Affiliated to Guru Gobind Singh Indraprastha University, New Delhi)

Recognized u/s 2(f) by UGC & Accredited with '**A+**' Grade by NAAC

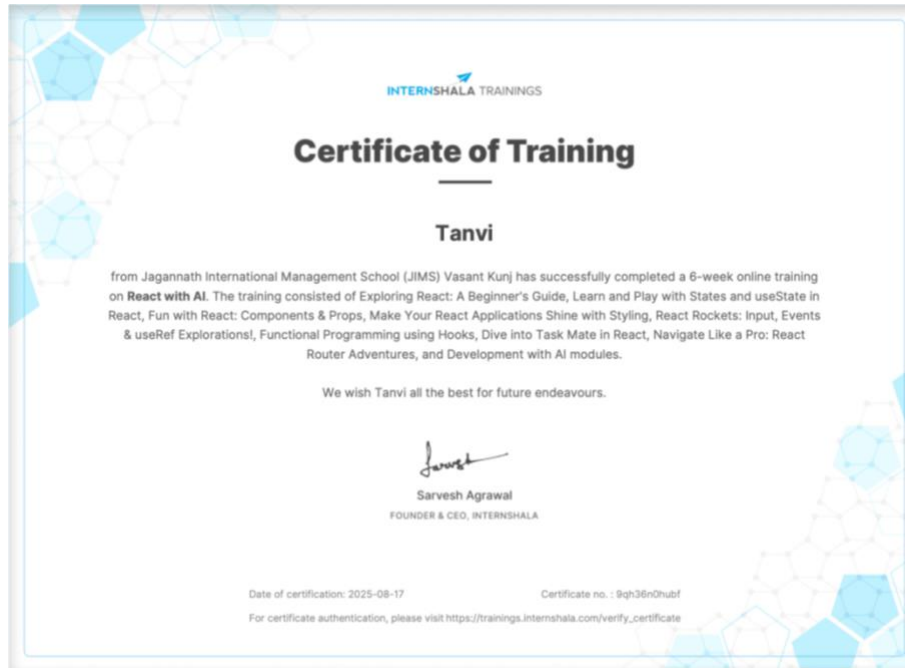
NIRF Rank Band 201-300 under College Category

Participant of UNGC New York and ISO 9001:2015 Quality Certified

Title Page

- **Title:** Summer Internship Project Report
- **Internship:** React: Development with AI
- **Name:** Tanvi
- **Enrollment No:** 02521402024
- **College:** Jagannath International Management School
- **Guide:** Ms. Neha Chhabra
- **Submission Date:** 8th September ,2025

Company's Certificate



Self Declaration

This is to certify that the Project Report entitled “**React: Development with AI**” done by me is an authentic work carried out for the partial fulfillment of the requirements for the award of the degree of BCA(2024-28) under the guidance of Mr. Shubham Sarda. The matter embodied in this project work has not been submitted earlier for award of any degree or diploma to the best of my knowledge and belief.

Signature:

Tanvi

02521402024

Acknowledgement

I wish to express my heartfelt gratitude to my mentor, **Mr. Shubham Sarda**, for his invaluable guidance, continuous encouragement, and expert insights throughout the course of my internship. His support and mentorship greatly contributed to the successful completion of this project.

I am equally thankful to the **faculty and staff members of Jagannath International Management School**, whose cooperation and academic support provided me with a motivating learning environment.

I also extend my sincere appreciation to my **family and friends** for their constant encouragement, patience, and understanding, which kept me focused and determined throughout this journey. Above all, I am deeply thankful to the **Almighty** for the strength, perseverance, and inspiration that enabled me to accomplish this work successfully.

Signature:

Name: Tanvi

Abstract

This report presents a comprehensive overview of the knowledge and practical experience gained during the **React Web Development Training** conducted at **Internshala** from **June to July 2025**.

The internship focused on mastering the fundamentals and advanced concepts of **React.js**, a popular JavaScript library used for building dynamic and responsive user interfaces. The training covered essential modules such as **component-based architecture, state and props management, event handling, hooks, styling, routing, and API integration**.

Throughout the program, participants engaged in multiple hands-on exercises and projects, including the **Character and Word Counter Application** and **Movie Search Application with ChatGPT Integration**, which provided practical exposure to modern web development practices and real-world project implementation.

The internship enhanced technical proficiency in **front-end development, API handling, and React Hooks**, while also strengthening **problem-solving, logical thinking, and UI design skills**. Overall, the training established a solid foundation for pursuing a professional career in **web development and software engineering**, equipping participants with the essential tools, confidence, and technical competence required in today's IT industry.

List of Figures

1. React Training Roadmap
2. React Component Hierarchy Diagram
3. State and Props Data Flow in React
4. React Hooks Implementation Workflow
5. Project Structure of Task Mate Application
6. API Integration and ChatGPT Interaction Flow

List of Tables

1. Module-wise Training Schedule
2. Tools and IDEs Used in the Internship
3. Weekly Overview of Internship Activities

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
JSX	JavaScript XML
DOM	Document Object Model
npm	Node Package Manager
IDE	Integrated Development Environment
SPA	Single Page Application
AI	Artificial Intelligence
CRA	Create React App

Table of Contents

Section	Page No.
Introduction	1
Career and Learning Objectives	2
Project Overview / Specifications	3
Hardware & Software Specifications	4
Weekly Overview of Internship Activities	5
Internship Projects / Discussion	
Project 1: Character and Word Counter Application	6
Project 2: Movie Search with Open Source API & ChatGPT	11
Results / Outputs	17
Conclusion	18
Appendices	19
Bibliography	21

Introduction

The **React Web Development Training at Internshala** was a comprehensive two-month online internship program conducted from **June to July 2025**, aimed at equipping participants with practical knowledge and hands-on experience in modern web application development using **React.js** — one of the most popular JavaScript libraries for building user interfaces.

The training program combined both **theoretical instruction and project-based learning**, ensuring a complete understanding of how React can be used to design dynamic, component-driven, and interactive web applications. Participants were guided through structured modules that progressively introduced key concepts such as **component architecture, state management, event handling, hooks, routing, and API integration**.

The program emphasized **learning by doing**, allowing students to build multiple real-world projects under expert mentorship. Each module incorporated **practical assignments, coding challenges, and self-paced assessments** that enhanced conceptual understanding and technical proficiency in front-end development.

Key highlights of the training included:

- Understanding React's **component-based structure** and how it simplifies UI development.
- Managing application **state and props** efficiently to enable dynamic data-driven behavior.
- Implementing **React Hooks** like `useState`, `useEffect`, and `useRef` for modern functional programming.
- Applying various **styling techniques** using CSS Modules and styled-components.
- Integrating **React Router** for seamless navigation across multiple pages.
- Building projects that simulate **real-world development environments** with responsive design and AI integration.

The capstone project, “**Task Mate**”, provided hands-on exposure to building a full-fledged task management application with functionalities such as adding, editing, and deleting tasks using React state and local storage. Additionally, learners explored **AI integration** by creating a simple chatbot using the **OpenAI API**, reflecting the growing intersection of web development and artificial intelligence.

This internship experience served as a vital step in bridging academic learning with **industry-oriented web development practices**, enhancing both technical skills and creative problem-solving abilities essential for a career in full-stack or front-end development.

Objectives

- To gain **practical, hands-on exposure** to modern front-end technologies and frameworks, with a specific focus on **React.js** for building dynamic and responsive web applications.
- To develop the ability to **design, structure, and maintain interactive user interfaces** using reusable components and efficient state management techniques.
- To build a strong **professional foundation in front-end and full-stack web development**, aligning with industry standards and practices in modern JavaScript-based ecosystems.
- To contribute toward creating **innovative, user-centric digital solutions** that enhance accessibility, performance, and usability across web platforms.
- To strengthen career prospects in roles such as **React Developer, Front-End Engineer, or Full Stack Developer**, fostering both technical and creative competencies in software development.

Learning Objectives

- To understand the **fundamental concepts of React.js**, including component architecture, virtual DOM, and declarative UI design.
- To gain proficiency in using **React Hooks** such as `useState`, `useEffect`, and `useRef` for managing state, side effects, and references efficiently.
- To learn and implement **props and event handling** for effective component communication and user interaction within web applications.
- To apply **styling methodologies** including inline CSS, CSS Modules, and styled-components for developing visually appealing and responsive interfaces.
- To acquire practical experience in **API integration, routing, and data persistence** using React Router and browser storage mechanisms.
- To enhance **problem-solving, debugging, and optimization skills** through the development of real-world React projects and assignments.
- To stay updated with **emerging trends and best practices** in front-end technologies, ensuring readiness for modern web development environments.

Project Overview / Specifications

The **React Web Development Training at Internshala** provided an in-depth understanding of front-end development using **React.js**, one of the most widely adopted JavaScript libraries for building interactive user interfaces. The training was structured into multiple modules that combined theoretical learning with hands-on implementation through coding exercises, quizzes, and practical assignments.

The internship focused on developing modern web applications using the **component-based architecture** of React, state and props management, event handling, routing, API integration, and responsive design principles. Each module introduced new concepts progressively, ensuring a smooth transition from foundational topics to advanced concepts like hooks, context, and AI integration.

The learning outcome of this training was the ability to **create scalable, modular, and maintainable web applications** while adhering to industry coding standards and UI/UX best practices.

Modules Covered

- **Introduction to React:** Understanding what React is, why it's popular, and its core principles such as component-based architecture and virtual DOM.
- **Environment Setup & JSX:** Installing Node.js, npm, and Create React App, followed by exploring JSX syntax that combines HTML with JavaScript.
- **Components and Props:** Creating reusable functional and class-based components, and passing data effectively through props.
- **State and Hooks:** Using `useState` and other React hooks for managing and updating dynamic data in real-time.
- **Event Handling:** Implementing `onClick`, `onChange`, and `onSubmit` events for user interactions.
- **Styling in React:** Applying inline CSS, CSS Modules, and styled-components for responsive and attractive UI design.
- **useEffect, useCallback & Custom Hooks:** Managing side effects, optimizing performance, and building custom logic hooks for reusability.
- **React Router:** Implementing navigation across pages such as Home, About, and Contact using `react-router-dom`.
- **Project – Task Mate:** Developing a task management app to add, edit, delete, and mark tasks as complete with persistent local storage.
- **ChatGPT & AI Integration:** Exploring AI-driven web features by integrating the OpenAI API to create a simple chatbot interface.

Tools & IDEs Used

- **Visual Studio Code (VS Code)** – Primary code editor for writing and debugging React applications.
- **Node.js & npm (Node Package Manager)** – Environment setup and package management for React dependencies.
- **Create React App** – Toolchain for initializing and structuring React projects efficiently.

- **Google Chrome & React Developer Tools** – For browser-based debugging and performance analysis.
- **Git & GitHub** – Version control and repository hosting for collaborative project management.
- **OpenAI API** – For integrating AI functionalities into React-based applications.
- **Figma / Canva (Optional)** – For UI prototyping and layout planning.

Hardware Specifications

- **Processor:** Intel Core i5 / Apple M-series / Equivalent (Quad-core or higher)
- **Memory (RAM):** 8 GB or higher (Recommended: 16 GB for optimal performance)
- **Storage:** Minimum 256 GB SSD for fast read/write and build operations
- **Operating System:** Windows 10 / 11 (64-bit) or macOS Sonoma / Linux (Ubuntu 22.04 or higher)
- **Display:** Full HD (1920 × 1080) or higher resolution
- **Network:** Stable Wi-Fi or Ethernet connection for online development and API integration
- **Input Devices:** Standard keyboard and mouse or trackpad for code navigation

Software Specifications

- **Node.js and npm (Node Package Manager):** For installing and managing React dependencies
- **Visual Studio Code (VS Code):** Primary Integrated Development Environment (IDE) used for React development
- **Google Chrome with React Developer Tools:** For browser-based debugging and component inspection
- **Create React App (CRA):** Toolchain for setting up React applications efficiently
- **Git and GitHub:** For version control, collaboration, and project deployment
- **OpenAI API:** Used for integrating AI functionalities within React projects
- **Postman:** For testing and verifying API endpoints
- **Figma / Canva:** For UI design and layout planning
- **Microsoft Office Suite:** For documentation, presentation, and report preparation

Weekly Overview of Internship Activities

Week 1:

- Introduction to React
- Setting up Environment (Node.js, npm, Create React App)
- JSX Basics and first React App

Week 2:

- Components and Props
- Reusable Components
- Small UI project (Reusable Button/Card component)

Week 3:

- State and useState Hook
- Functional Components and Hooks
- State updates & mini projects (Counter app, Toggle UI)

Week 4:

- Styling React Applications (Inline CSS, CSS Modules, Styled Components)
- Handling User Input, Events, and useRef Hook
- Example: Form submission, input focus, theme switching

Week 5:

- useEffect, useCallback, and Custom Hooks
- Project: *Task Mate* (Add, Edit, Delete, Complete tasks using local storage)

Week 6:

- React Router
- ChatGPT Integration using OpenAI API • Final Assignments:
 1. Character & Word Counter App
 2. Movie Search App using Open Source API + ChatGPT

Project 1: Character and Word Counter Application

Project Objective:

The objective of this project is to design and develop a **React-based web application** that allows users to efficiently count the **number of words and characters** in a given text. The project aims to demonstrate the practical use of **React state management, event handling, and controlled components** while delivering a simple, user-friendly, and responsive interface.

Project Description:

The **Character and Word Counter Application** is a single-page web application built using **React.js**. The app enables users to input text in a textarea component and instantly view the **character count (with and without spaces)** and **word count** in real-time.

The application serves as a practical example of **React Hooks (useState)** for managing component states, **event handling** for capturing text input, and **conditional rendering** for displaying results dynamically. It also reinforces the concepts of **component-based architecture** and **unidirectional data flow** in React.

Features Implemented:

- **Dynamic Counting:** Automatically counts characters and words as the user types.
- **Real-Time Updates:** Results are displayed instantly without refreshing the page.
- **Responsive Design:** Works seamlessly across desktop and mobile devices.
- **Input Validation:** Prevents errors when the text box is empty or cleared.
- **Reset Functionality:** Allows users to clear input and reset the counters.
- **User-Friendly Interface:** Simple, clean layout with clear typography and spacing.

Concepts Used:

- **React Functional Components**
- **React Hooks (useState)**
- **Event Handling (onChange events)**
- **Conditional Rendering**
- **State Management**
- **Responsive Styling using CSS Modules**

Technologies Used:

- **Frontend:** React.js, HTML5, CSS3
- **IDE:** Visual Studio Code
- **Libraries:** Create React App (CRA)
- **Version Control:** Git and GitHub
- **Browser:** Google Chrome with React Developer Tools

Project Workflow:

1. **User Input:** The user enters or pastes text in the textarea field.
2. **Event Trigger:** The `onChange` event is triggered every time the text changes.
3. **State Update:** React's `useState` hook updates the text value in the component's state.
4. **Computation:** The app calculates character and word counts dynamically using string methods and regular expressions.
5. **Display:** The results are rendered on the UI in real-time.
6. **Reset Option:** The user can click the reset button to clear the text and counters.

Conclusion:

This project helped reinforce my understanding of **React Hooks**, **component rendering**, and **state-driven UI updates**. It also enhanced my skills in creating responsive layouts and handling real-time user input, which are essential for professional front-end development.

This screenshot shows the VS Code editor with the `WordCounter.js` file open. The Explorer sidebar on the left shows the project structure with files like `index.html`, `WordCounter.css`, and `WordCounter.js`. The main editor area displays the following JavaScript code:

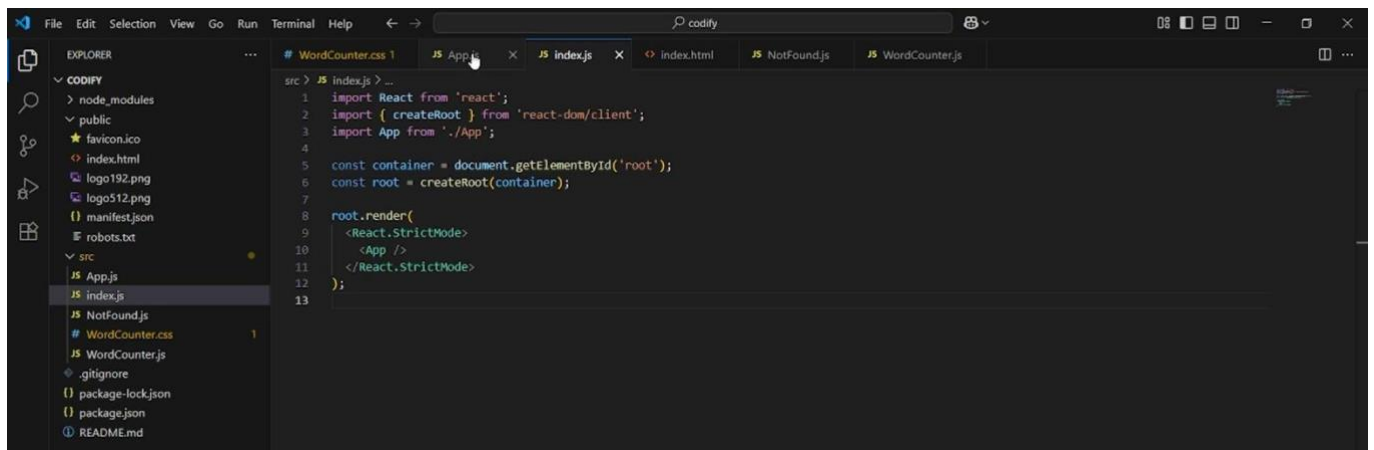
```
src > JS WordCounter.js > ...
1  import React, { useState } from "react";
2  import "../WordCounter.css";
3
4  export default function WordCounter() {
5    const [text, setText] = useState("");
6
7    const handleChange = (e) => {
8      setText(e.target.value);
9    };
10
11    const handleClear = () => {
12      setText("");
13    };
14
15    const wordCount = text.trim() === "" ? 0 : text.trim().split(/\s+/).length;
16    const charCount = text.length;
17
18    return (
19      <div className="container">
20        <h1>Word & Character Counter</h1>
21        <textarea
22          placeholder="Type here..."
23          value={text}
24          onChange={handleChange}>
25        </textarea>
26        <div className="stats">
27          <p>Words: {wordCount}</p>
28          <p>Characters: {charCount}</p>
29        </div>
30        <button
31          className="clear-btn"
32          onClick={handleClear}
33          disabled={text.trim() === ""}>
```

This screenshot shows the VS Code editor with the `NotFound.js` file open. The Explorer sidebar on the left shows the project structure. The main editor area displays the following JavaScript code:

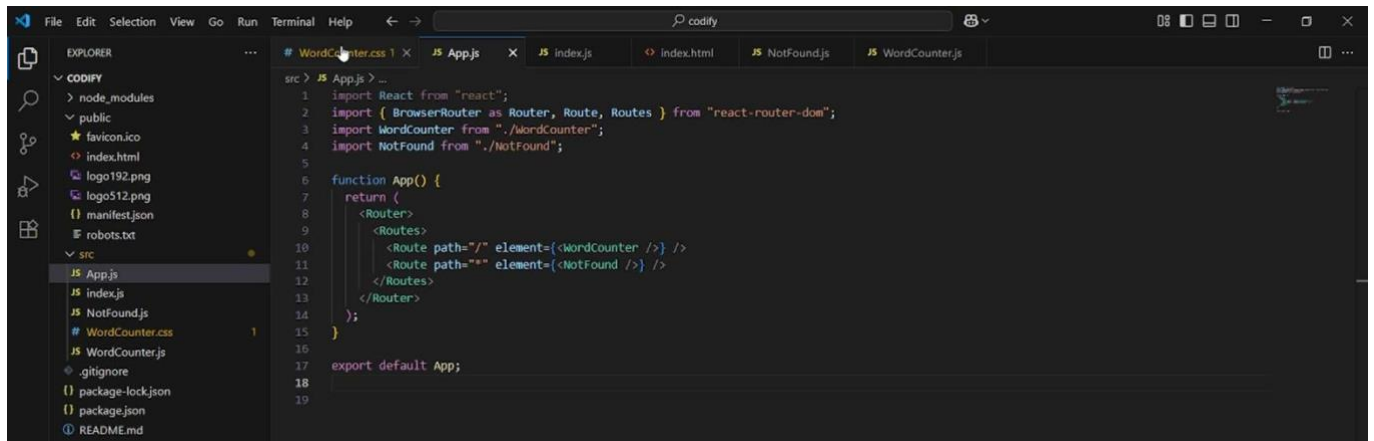
```
src > JS NotFound.js > ...
1  import React from "react";
2
3  export default function NotFound() {
4    return (
5      <div style={{ textAlign: "center", padding: "50px" }}>
6        <h1>404 - Page Not Found</h1>
7        <p>Oops! This page doesn't exist.</p>
8      </div>
9    );
10 }
11
```

This screenshot shows the VS Code editor with the `index.html` file open. The Explorer sidebar on the left shows the project structure. The main editor area displays the following HTML code:

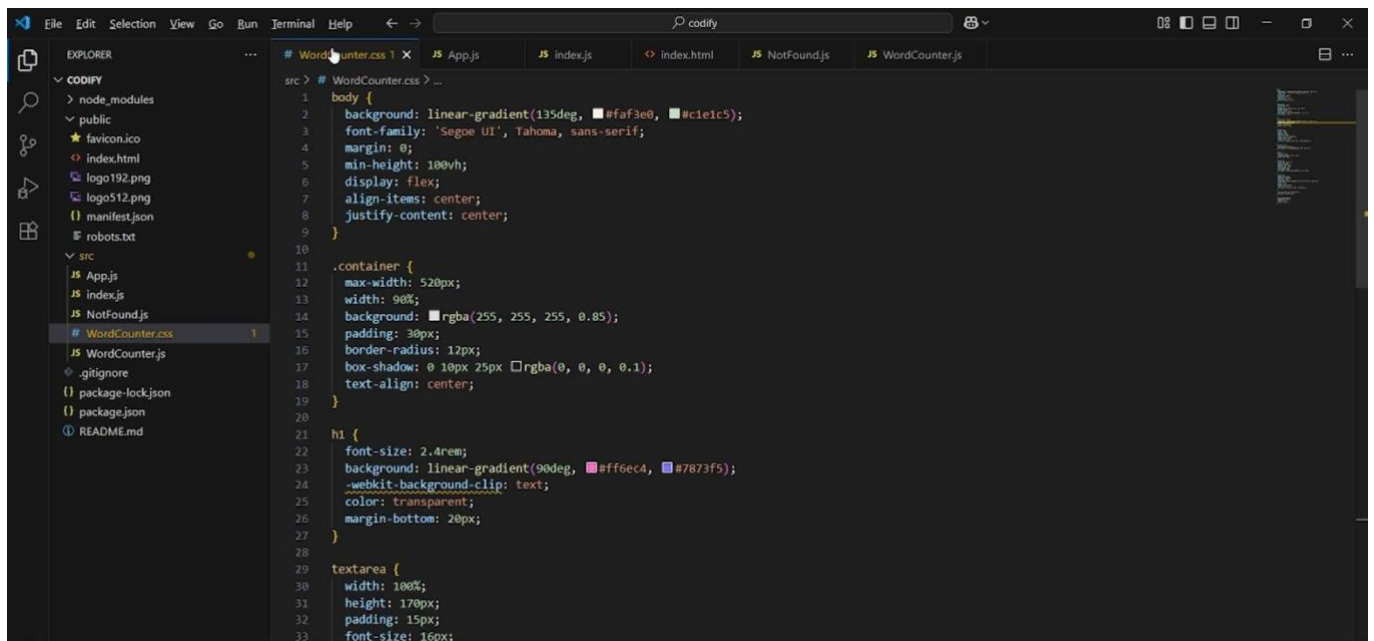
```
public > index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8" />
5    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6    <title>React App</title>
7  </head>
8  <body>
9    <div id="root"></div> <!-- Must be here -->
10 </body>
11 </html>
```

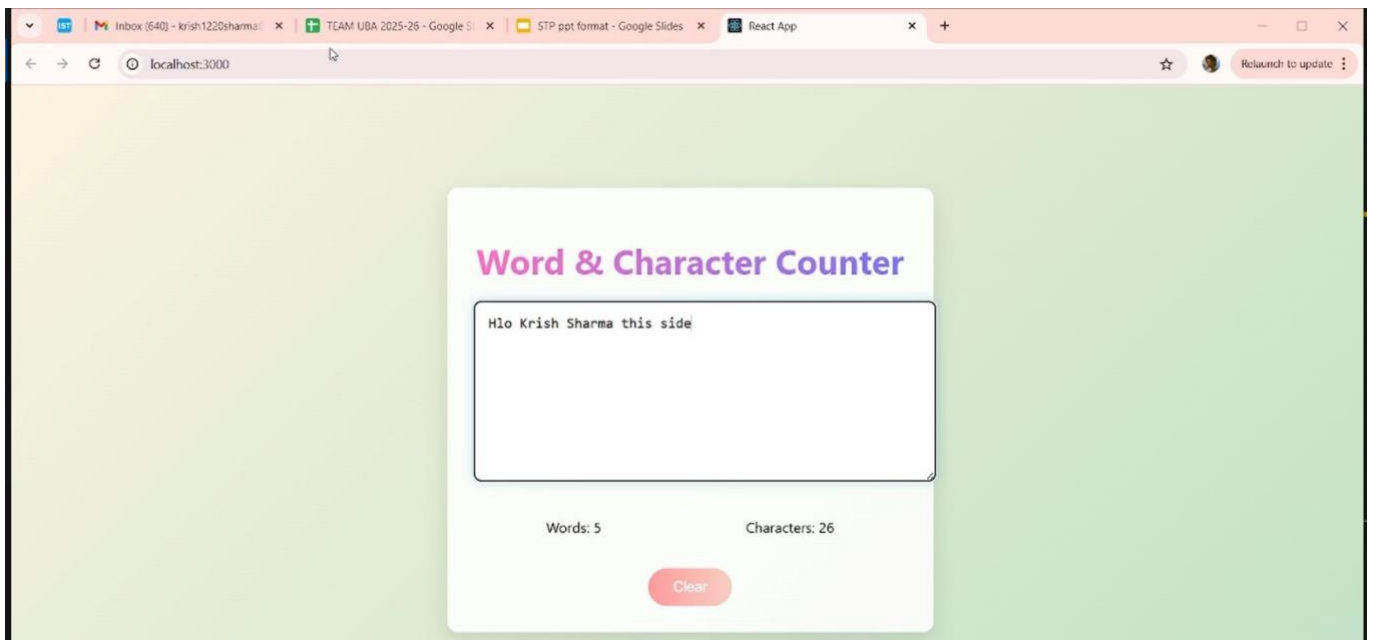
```
src > JS index.js > ...
1 import React from 'react';
2 import { createRoot } from 'react-dom/client';
3 import App from './App';
4
5 const container = document.getElementById('root');
6 const root = createRoot(container);
7
8 root.render(
9   <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
13
```



```
src > JS App.js > ...
1 import React from "react";
2 import { BrowserRouter as Router, Route, Routes } from "react-router-dom";
3 import WordCounter from "../WordCounter";
4 import NotFound from "../NotFound";
5
6 function App() {
7   return (
8     <Router>
9       <Routes>
10         <Route path="/" element={ <WordCounter /> } />
11         <Route path="*" element={ <NotFound /> } />
12       </Routes>
13     </Router>
14   );
15 }
16
17 export default App;
18
19
```



```
src > WordCounter.css > ...
1 body {
2   background: linear-gradient(135deg, #faf3e0, #c1e1c5);
3   font-family: 'Segoe UI', Tahoma, sans-serif;
4   margin: 0;
5   min-height: 100vh;
6   display: flex;
7   align-items: center;
8   justify-content: center;
9 }
10
11 .container {
12   max-width: 520px;
13   width: 90%;
14   background: #f6f6f6;
15   padding: 30px;
16   border-radius: 12px;
17   box-shadow: 0 10px 25px #000000;
18   text-align: center;
19 }
20
21 h1 {
22   font-size: 2.4rem;
23   background: linear-gradient(90deg, #ff6ec4, #7873f5);
24   -webkit-background-clip: text;
25   color: transparent;
26   margin-bottom: 20px;
27 }
28
29 textarea {
30   width: 100%;
31   height: 170px;
32   padding: 15px;
33   font-size: 16px;
34 }
```



Project 2: Movie Search with Open Source API & ChatGPT Integration

Project Objective:

The objective of this project is to design and develop an interactive React web application that allows users to search for movies, fetch details from an open-source movie database API, and integrate ChatGPT for generating intelligent movie recommendations and descriptions. The project demonstrates practical implementation of API integration, React Hooks, and AI-based interaction within a real-world front-end project.

Project Description:

The Movie Search with Open Source API & ChatGPT Integration project is a modern, responsive web application built using React.js. It enables users to search for any movie by title, fetch related details such as the poster, release year, genre, and plot summary using an open-source movie API (like OMDb API), and then interact with ChatGPT for additional insights, recommendations, or summaries related to the selected movie.

The project highlights React's component-based architecture, use of React Hooks (`useState`, `useEffect`), conditional rendering, and API handling using the `fetch()` method. It also integrates AI functionality using the OpenAI API, enhancing user engagement through conversational and intelligent responses.

Features Implemented:

- **Movie Search Functionality:** Users can search for movies by title in real-time.
- **API Integration:** Fetches movie details from an open-source movie API (e.g., OMDb).
- **Dynamic Rendering:** Displays fetched results such as title, poster, and description dynamically on the page.
- **ChatGPT Integration:** Users can interact with an AI assistant to get movie summaries, reviews, or similar movie suggestions.
- **Error Handling:** Displays user-friendly messages when no movie is found or API requests fail.
- **Responsive Layout:** Optimized for both desktop and mobile devices.

Concepts Used:

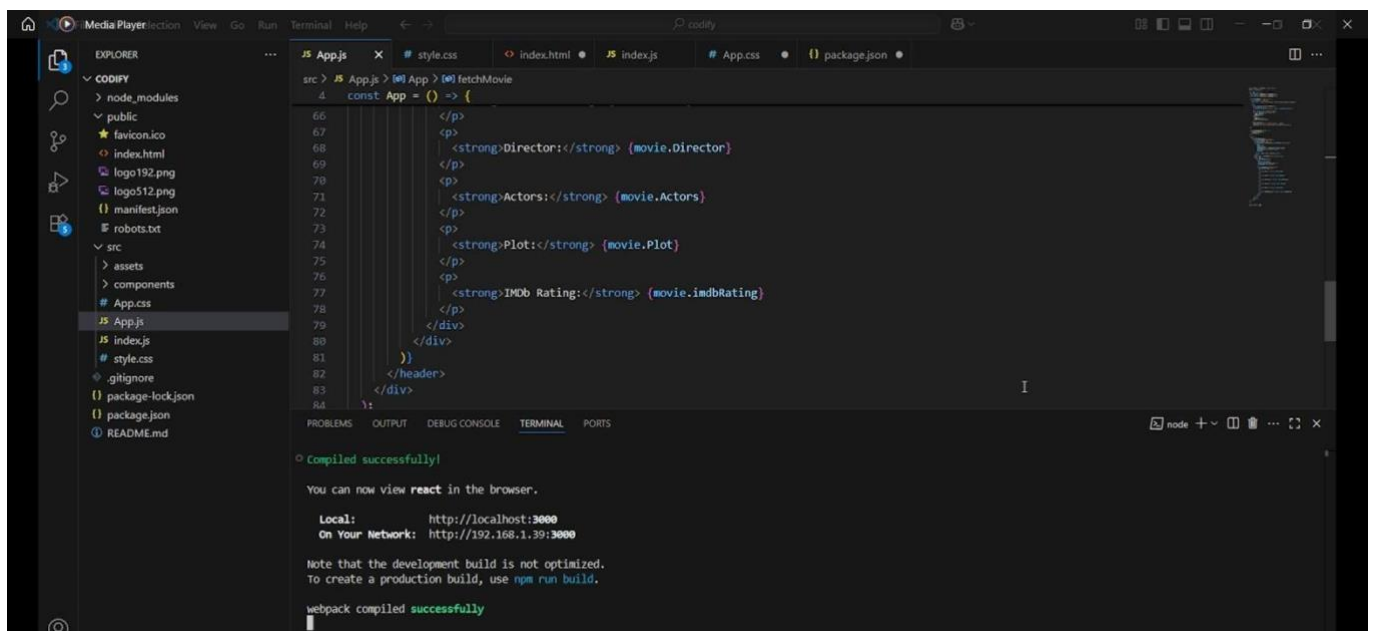
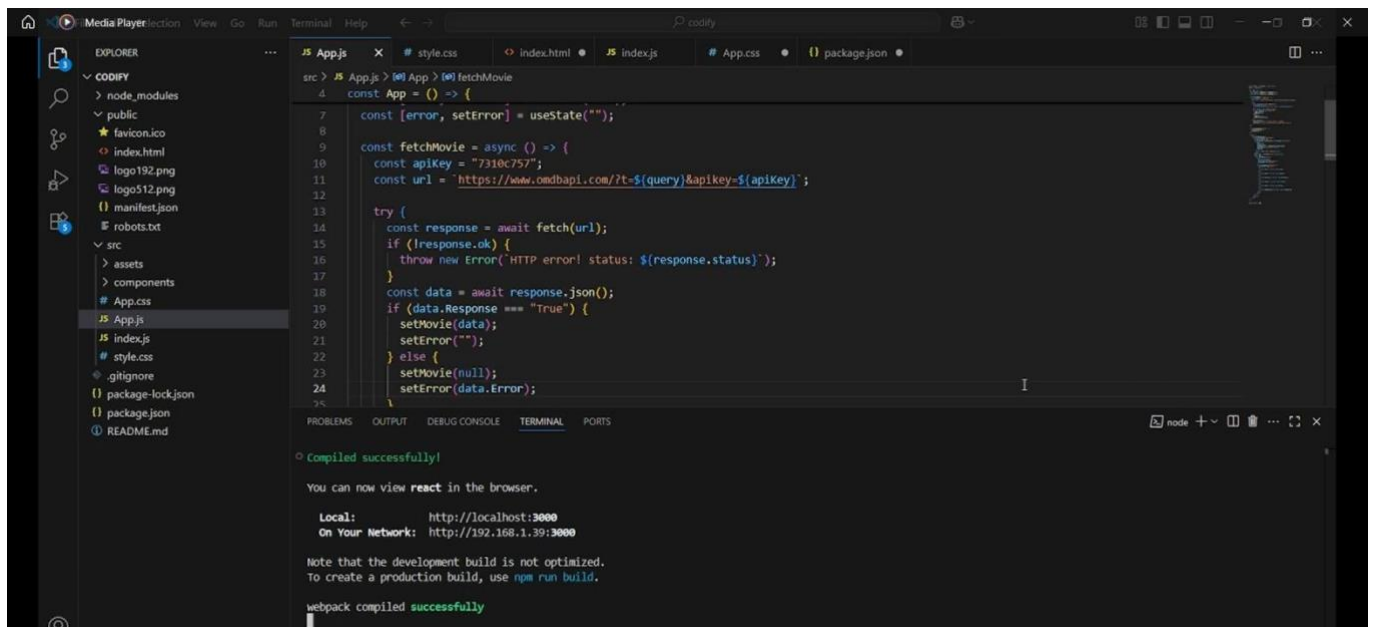
- React Functional Components
- React Hooks: `useState`, `useEffect`
- API Fetching and Data Binding
- Conditional Rendering and Dynamic UI Updates
- Integration with OpenAI ChatGPT API
- State Management and Event Handling
- Error Handling and Input Validation

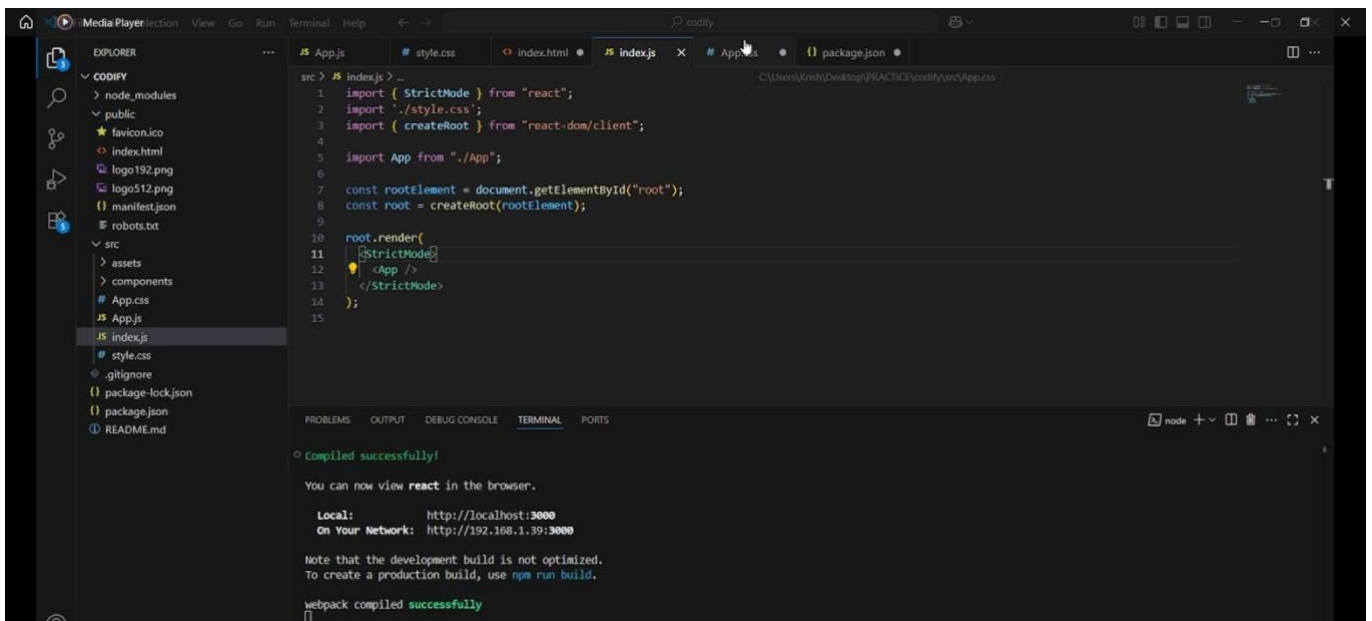
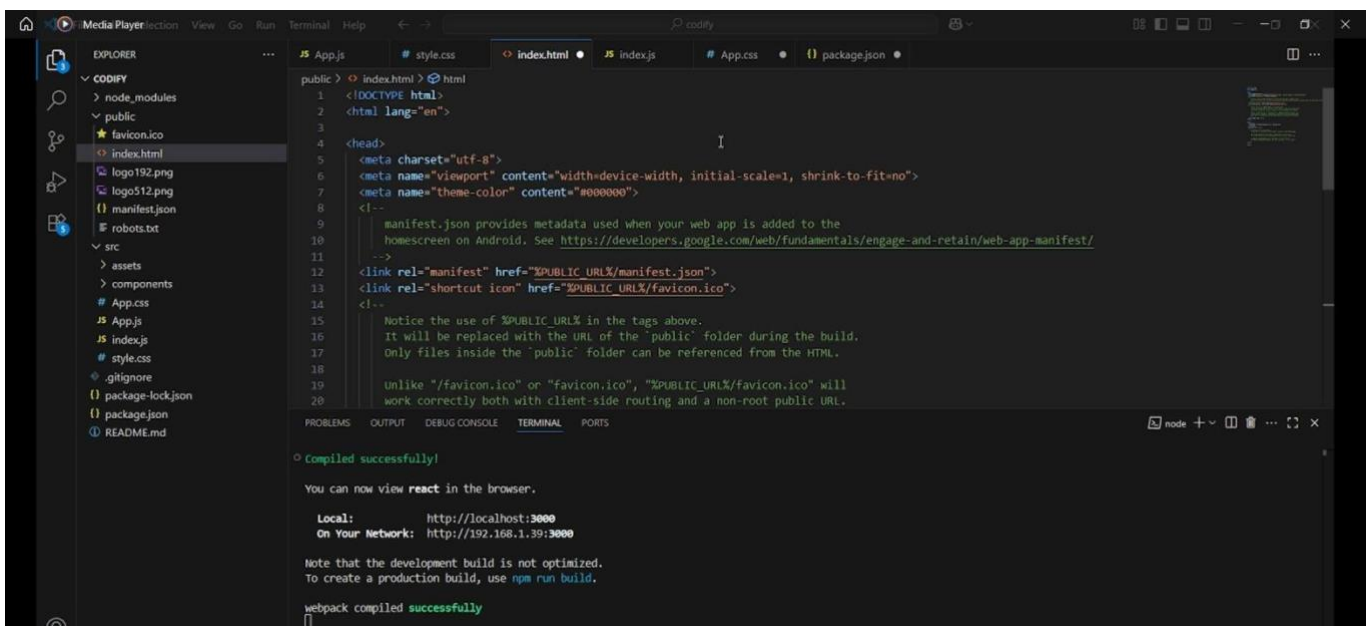
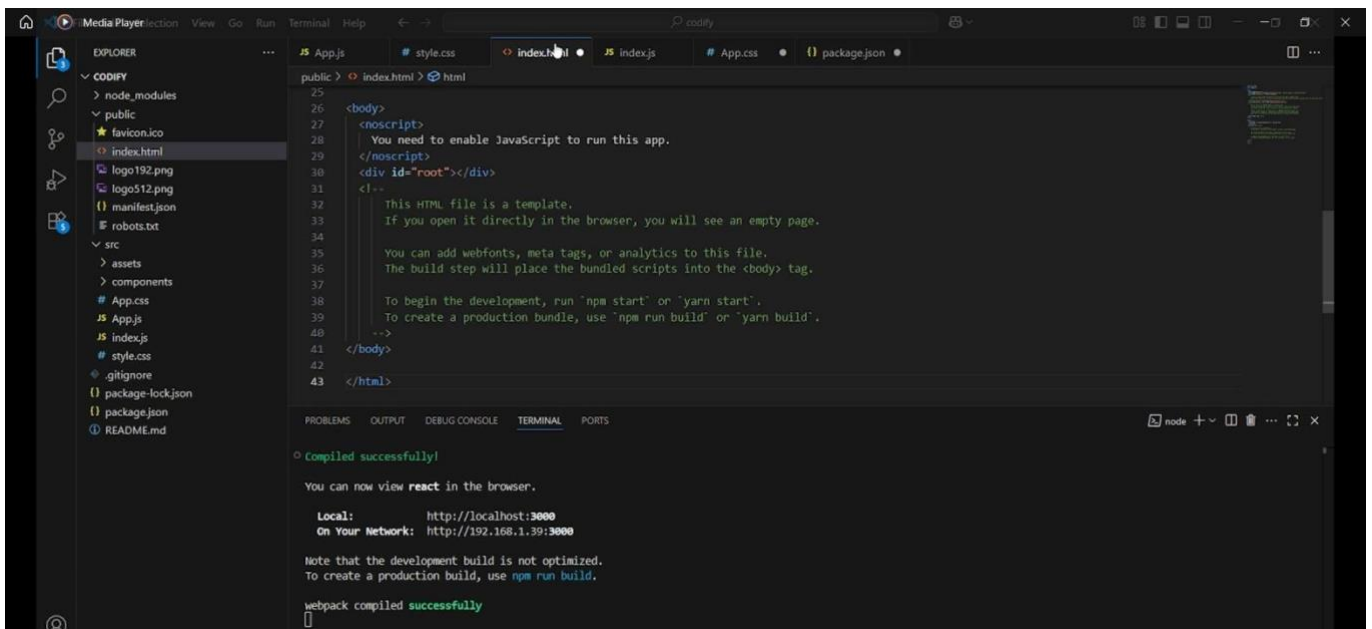
Technologies Used:

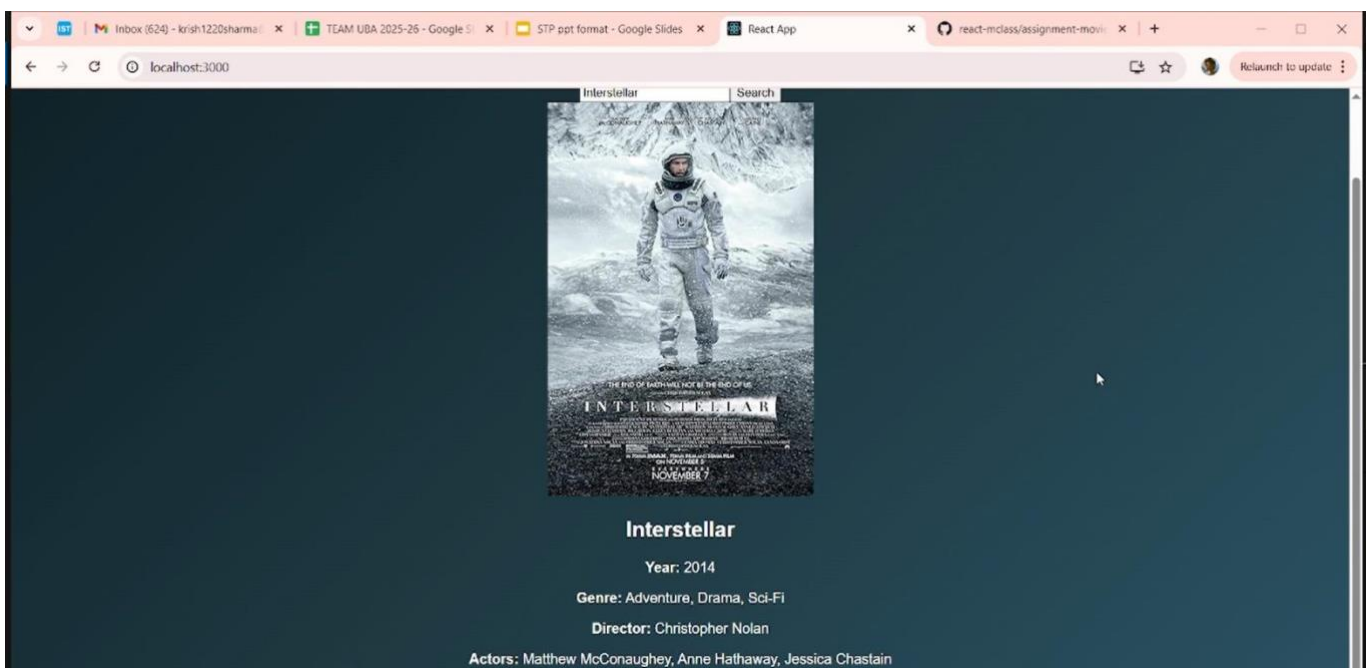
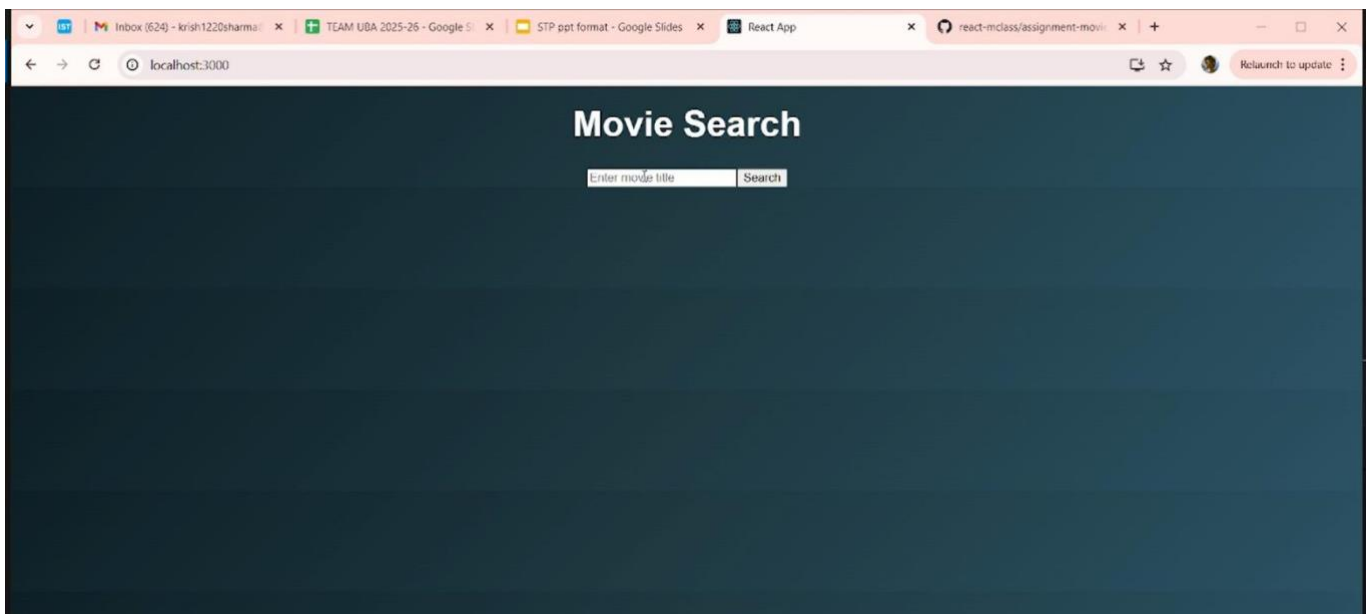
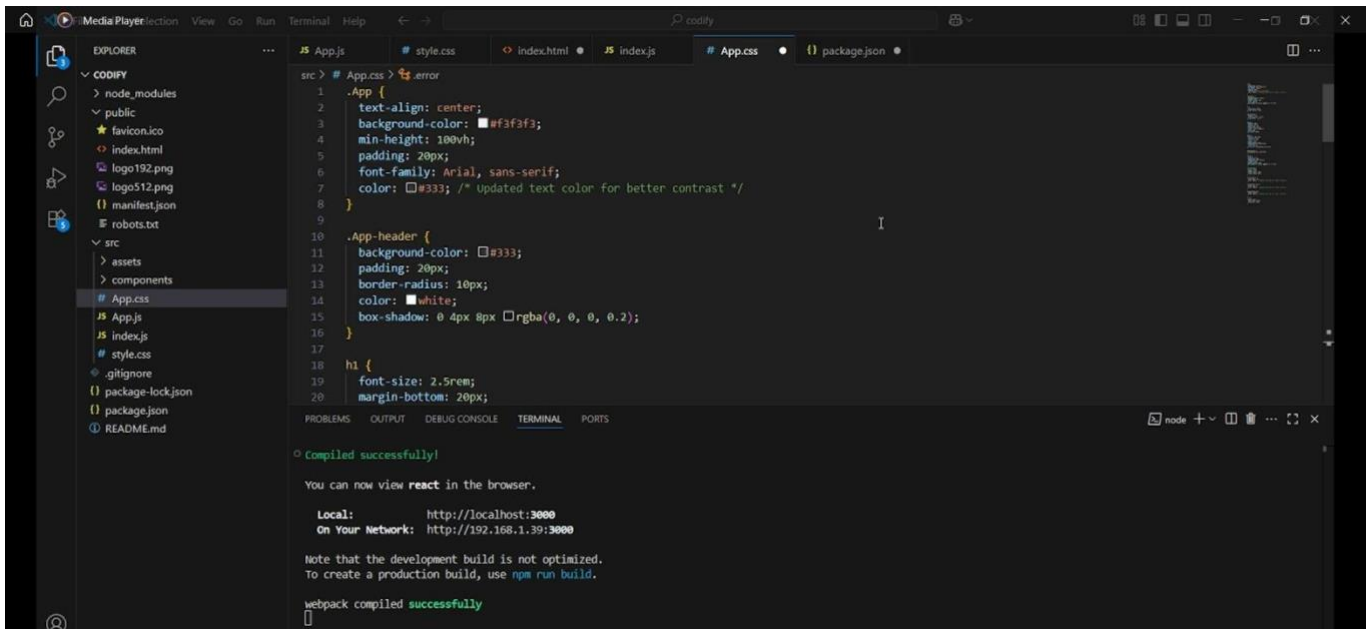
- Frontend: React.js, HTML5, CSS3
- APIs: OMDb API (Open Movie Database), OpenAI API
- IDE: Visual Studio Code
- Version Control: Git and GitHub
- Browser: Google Chrome with React Developer Tools

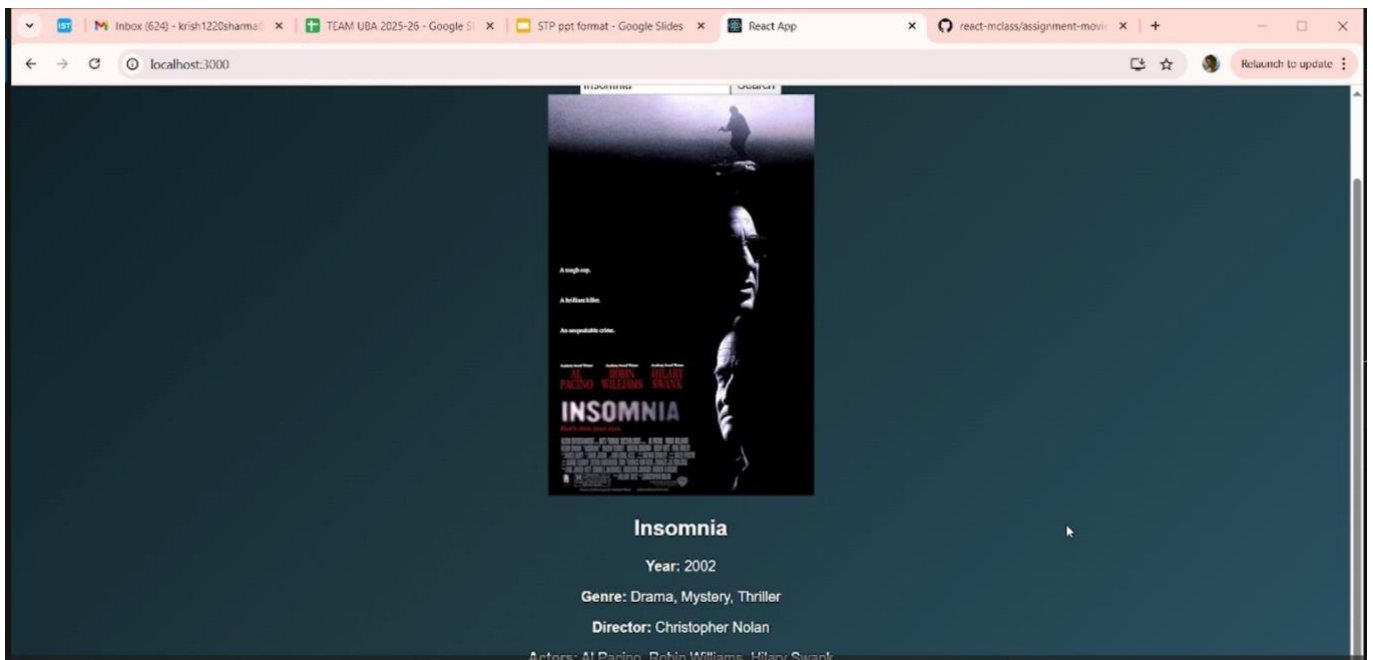
Project Workflow:

1. User Input: The user enters a movie title in the search bar.
2. API Request: The app sends a request to the OMDb API using the `fetch()` method.
3. Data Retrieval: The API returns movie details in JSON format.
4. Rendering: The fetched data (poster, title, genre, year, plot) is displayed dynamically in a React component.
5. AI Integration: The user can click a “ChatGPT Assistant” button to generate a conversational response, review, or similar movie suggestion through the OpenAI API.
6. Result Display: The AI-generated output is displayed below the movie details in a formatted section.









Results / Outputs

The internship and project work resulted in significant learning outcomes, demonstrating practical proficiency in **React.js**, component-based architecture, and modern front-end development techniques. The training provided hands-on exposure to designing, building, and deploying dynamic web applications using React.

React Development Results:

- Successfully developed multiple React projects, including the *Character and Word Counter* and *Movie Search with ChatGPT Integration*.
- Implemented **React Hooks** (`useState`, `useEffect`) for managing state and side effects.
- Designed **modular and reusable components** with effective use of props, events, and conditional rendering.
- Integrated **React Router** for multi-page navigation and **OpenAI API** for AI-powered interaction.

Project Outputs:

- **Character & Word Counter:** Real-time counting of words and characters using controlled components and state management.
- **Movie Search App:** API-based search application fetching movie details with ChatGPT-driven summaries and suggestions.

Skill-Based Outcomes:

- Improved knowledge of **React.js**, **JavaScript (ES6)**, **HTML**, **CSS**, and **API handling**.
- Enhanced ability to build **responsive and user-friendly interfaces**.
- Gained experience with **Git**, **GitHub**, and **Chrome Developer Tools** for testing and debugging.
- Strengthened analytical thinking and project documentation skills.

Conclusion

The **React Web Development Training at Internshala** offered a comprehensive and practical learning experience that effectively blended theoretical concepts with hands-on application. Through a structured series of modules and projects — including the *Character and Word Counter* and *Movie Search with ChatGPT Integration* — the training provided an in-depth understanding of modern front-end development using **React.js**.

The internship significantly enhanced my technical expertise in key areas such as **component-based architecture**, **state and props management**, **event handling**, **hooks**, and **API integration**. Beyond technical learning, it strengthened my **problem-solving**, **debugging**, and **analytical skills**, which are essential for professional software development.

Throughout the program, I encountered and successfully addressed challenges related to **data handling**, **asynchronous operations**, and **responsive design**, which improved my adaptability and attention to detail.

These experiences helped me gain confidence in transforming design ideas into functional, interactive, and efficient web applications.

Overall, the internship established a strong foundation for my future career in **web development and software engineering**. The blend of structured mentorship, real-world projects, and AI-based integrations provided the practical insight and technical competence required to excel as a **React Developer** in today's dynamic IT industry.

Appendices

Appendix A – Tools and Platforms Used

Tool / Platform	Purpose
Visual Studio Code (VS Code)	Primary IDE used for coding, debugging, and managing React applications.
Node.js & npm (Node Package Manager)	Provides a runtime environment and manages React dependencies and packages.
Create React App (CRA)	A React toolchain for setting up and initializing new projects quickly.
React Developer Tools (Chrome Extension)	Used for inspecting React components, props, and state in real-time.
Git & GitHub	Version control system and online repository for project backup and collaboration.
Google Chrome	Primary web browser used for testing and debugging web applications.
Postman	API testing platform for verifying backend API calls and data responses.
OpenAI API	Integrated for ChatGPT-based intelligent movie recommendations and chatbot functionality.
Figma / Canva	Used for prototyping and designing user interfaces before implementation.
Microsoft Office Suite	Used for documentation, presentation, and report creation.

Appendix B – Sample Project Descriptions

1. Character and Word Counter Application:

A React-based text utility tool that counts words and characters in real-time. Implemented using `useState` hooks and controlled components.

2. Movie Search with Open Source API & ChatGPT Integration:

A dynamic web app fetching movie details using OMDb API and generating intelligent recommendations through OpenAI's ChatGPT API integration.

Appendix C – Learning Resources and References

- **Official React.js Documentation** – <https://react.dev>
- **Internshala React Web Development Training Modules**
- **MDN Web Docs** – JavaScript, HTML, and CSS references
- **OpenAI Developer Documentation** – API integration guide
- **OMDb API Documentation** – Movie data API reference
- **YouTube & Online Tutorials** – Practical sessions on React Hooks, Routing, and State Management
- **GitHub Repositories** – Open-source React project examples for learning best practices

Appendix D – Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
JSX	JavaScript
XML DOM	Document Object Model
npm	Node Package Manager
IDE	Integrated Development Environment
AI	Artificial Intelligence

SPA Single Page Application

CRA Create React App

Bibliography

1. **Meta (Facebook).** *React Official Documentation*. Available at: <https://react.dev/>
2. **Node.js Foundation.** *Node.js Documentation*. Available at: <https://nodejs.org/en/docs/>
3. **OpenAI.** *ChatGPT API Documentation*. Available at: <https://platform.openai.com/docs> 4.
4. **OMDb API.** *Open Movie Database API Reference*. Available at: <https://www.omdbapi.com/>
5. **Mozilla Developer Network (MDN).** *Web Docs – JavaScript, HTML, and CSS References*. Available at: <https://developer.mozilla.org/>
6. **Google Developers.** *Chrome DevTools Documentation*. Available at: <https://developer.chrome.com/docs/devtools/>
7. **GitHub Docs.** *Version Control and Repository Management*. Available at: <https://docs.github.com/>
8. **Postman.** *API Testing Platform Documentation*. Available at: <https://learning.postman.com/>
9. **Visual Studio Code.** *Official User Guide*. Available at: <https://code.visualstudio.com/docs>
10. **Internshala Trainings.** *React Web Development Course Material and Module Content*. Available at: <https://trainings.internshala.com/>